1. **Describe unary operation with example.**

   A **unary operation** is an operation that involves only **one operand**. It performs a task using a **single value** and returns a result. In programming, unary operators are used to manipulate or evaluate individual variables.

   Types of Unary Operators in C:

   1. **Unary plus (+)** – Indicates a positive value (rarely used).
   2. **Unary minus (−)** – Negates the value.
   3. **Increment (++)** – Increases the value by 1.
   4. **Decrement (−−)** – Decreases the value by 1.
   5. **Logical NOT (!) **– Reverses the logical state (true ↔ false).
   6. **Bitwise NOT (~)** – Inverts all bits of the value.
   7. **Sizeof** – Returns the size of a data type or variable.

   Example:

```
#include <stdio.h>
int main() {
    int a = 5;

    printf("Unary minus: %d\n", -a);    // Output: -5
    printf("Increment: %d\n", ++a);     // Output: 6
    printf("Decrement: %d\n", --a);     // Output: 5
    printf("Logical NOT: %d\n", !a);    // Output: 0 (since a is
non-zero)

    return 0;
}
```

2. **Discuss the following: Bitwise operator & Logical operator**

   Bitwise Operator: Bitwise operators are used to perform operations on **binary representations** of integers. These operators work at the **bit level** and are mostly used in low-level programming, device control, and performance optimization.

   Common Bitwise Operators:

Operator	Description	Example (a = 5, b = 3)
&	Bitwise AND	a & b → 1
|	Bitwise OR	
^	Bitwise XOR	a ^ b → 6
~	Bitwise NOT	~a → -6 (in 2's comp)
<<	Left shift	a << 1 → 10
>>	Right shift	a >> 1 → 2

Logical Operator: Logical operators are used to combine or invert Boolean expressions. They evaluate expressions based on true (1) or false (0) results.

Common Logical Operators:

Operator	Description	Example
&&	Logical AND	(a > 0 && b > 0) → 1
!	Logical NOT	!(a > 0) → 0

3. **Elaborate on the following: Assignment operator Arithmetic operator.**

Assignment Operator: The assignment operator is used to assign a value to a variable. The most common is the equal sign (=).

Types of Assignment Operators:

Operator	Description	Example
=	Simple assignment	a = 10;
+=	Add and assign	a += 5; → a = a + 5;
-=	Subtract and assign	a -= 2;
*=	Multiply and assign	a *= 3;
/=	Divide and assign	a /= 2;
%=	Modulus and assign	a %= 4;

Arithmetic Operator: Arithmetic operators are used to perform basic mathematical operations such as addition, subtraction, multiplication, etc

Operator	Description	Example (a = 10, b = 3)
+	Addition	a + b → 13
-	Subtraction	a - b → 7
*	Multiplication	a * b → 30
/	Division	a / b → 3
%	Modulus (remainder)	a % b → 1

4. **Explain Logical operator with an example**

Definition:

Logical operators are used to combine two or more conditions or to reverse the logic of a condition. These operators return a Boolean result:

- 1 (true) if the condition is met
- 0 (false) otherwise.

Logical operators are mainly used in decision-making statements like if, while, etc.

Types of Logical Operators in C:

Operator	Name	Description
&&	Logical AND	Returns true if both conditions are true
||	Logical OR	Returns true if one of the conditions is true
!	Logical NOT	Reverses the result: true becomes false and vice versa

Example:

```
#include <stdio.h>
int main() {
    int a = 5, b = 10;

    if (a > 0 && b > 0)
        printf("Both are positive\n");

    if (a < 0 || b > 0)
        printf("At least one is positive\n");

    if (!(a < 0))
        printf("a is not negative\n");

    return 0;
}
```

Output:

```
Both are positive
At least one is positive
a is not negative
```

5. **Summarize Increment and Decrement operator**

Increment Operator (++)

- Increases the value of a variable by 1.
- Two types:
    - Pre-increment (++a): Increments first, then uses the value.
    - Post-increment (a++): Uses the value, then increments.

Example:

```
int a = 5;
printf("%d", ++a); // Output: 6
printf("%d", a++); // Output: 6 (then a becomes 7)
```

Decrement Operator (--)

- Decreases the value of a variable by 1.
- Two types:
    - **Pre-decrement (--a)**: Decrements first, then uses the value.
    - **Post-decrement (a--)**: Uses the value, then decrements.

Example:

```
int a = 5;
```

```
printf("%d", --a); // Output: 4
printf("%d", a--); // Output: 4 (then a becomes 3)
```

6. **Demonstrate the difference between "break" and "continue" with an example.**

Feature	break	continue
Purpose	Exits the loop entirely	Skips to the next iteration
Affects	Whole loop or switch	Only current iteration
Common Use	Exiting loops early	Skipping specific values in loops

Example Program:

```
#include <stdio.h>
int main() {
    int i;

    // Using break
    printf("Using break:\n");
    for (i = 1; i <= 5; i++) {
        if (i == 3)
            break;
        printf("%d ", i);
    }

    // Using continue
    printf("\nUsing continue:\n");
    for (i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
        printf("%d ", i);
    }

    return 0;
}
```

Output:

```
Using break:
1 2

Using continue:
1 2 4 5
```

7. **Contrast prefix and postfix with a small example**

Prefix and postfix are two forms of increment (++) and decrement (--) operators used to modify the value of a variable by 1.

Feature	Prefix (++a / --a)	Postfix (a++ / a--)
Operation order	Increments/Decrements first, then uses the value	Uses the value first, then increments/decrements
Usage context	Common in loops and conditional checks	Often used where previous value is needed before change

Example Code:

```
#include <stdio.h>
int main() {
    int a = 5, b;

    b = ++a;  // Prefix: a becomes 6, then b = 6
    printf("Prefix: a = %d, b = %d\n", a, b);

    a = 5;
    b = a++;  // Postfix: b = 5, then a becomes 6
    printf("Postfix: a = %d, b = %d\n", a, b);

    return 0;
}
```

Output:

```
Prefix: a = 6, b = 6
Postfix: a = 6, b = 5
```

8. **Explain the binary operators in brief with a suitable example.**

Binary operators are operators that operate on two operands. They are widely used in arithmetic, relational, logical, and bitwise operations.

Types of Binary Operators:

Category	Operator Examples	Description
Arithmetic	+, -, *, /, %	Perform basic math operations
Relational	==, !=, >, <, >=, <=	Compare two values
Logical	&&, `	
Bitwise	&, `	, ^, <<, >>`
Assignment	=, +=, -=, *=	Assign values or expressions

Example:

```
#include <stdio.h>

int main() {
    int a = 10, b = 5;

    printf("Addition: %d\n", a + b);        // Arithmetic
    printf("Greater: %d\n", a > b);         // Relational
    printf("Logical AND: %d\n", a > 0 && b > 0);  // Logical
```

```
    printf("Bitwise AND: %d\n", a & b);     // Bitwise
    return 0;
}
```

Output:

```
Addition: 15
Greater: 1
Logical AND: 1
Bitwise AND: 0
```

9. **Compose a complete C program to convert a given temperature in Fahrenheit scale to its equivalent Centigrade scale.**

```
#include <stdio.h>

int main() {
    float fahrenheit, celsius;

    // Input temperature in Fahrenheit
    printf("Enter temperature in Fahrenheit: ");
    scanf("%f", &fahrenheit);

    // Conversion formula
    celsius = (fahrenheit - 32) * 5 / 9;

    // Output result
    printf("Temperature in Centigrade: %.2f°C\n", celsius);

    return 0;
}
```

Output:

```
Enter temperature in Fahrenheit: 98.6
Temperature in Centigrade: 37.00°C
```

10. **Compose a C program to calculate the gross salary (sum of basic salary, DA, HRA) of an employee where DA is 35% of basic salary, HRA is 15% of basic salary and basic salary is taken as input.**

```
#include <stdio.h>
int main() {
    float basic_salary, DA, HRA, gross_salary;

    // Input basic salary
    printf("Enter the basic salary: ");
    scanf("%f", &basic_salary);

    // Calculate DA and HRA
    DA = 0.35 * basic_salary;   // 35% of basic
    HRA = 0.15 * basic_salary;  // 15% of basic
```

```
    // Calculate gross salary
    gross_salary = basic_salary + DA + HRA;

    // Output the result
    printf("Gross Salary = %.2f\n", gross_salary);

    return 0;
}
```

Output:

```
Enter the basic salary: 10000
Gross Salary = 15000.00
```

11. **Compose a C program to find the simple interest and total amount where the principal amount is 3000, number of year is two and rate of interest is 10%.**

```
#include <stdio.h>
int main() {
    float principal = 3000, rate = 10, time = 2;
    float simple_interest, total_amount;

    // Calculate Simple Interest
    simple_interest = (principal * rate * time) / 100;

    // Calculate Total Amount
    total_amount = principal + simple_interest;

    // Output the results
    printf("Simple Interest = %.2f\n", simple_interest);
    printf("Total Amount = %.2f\n", total_amount);

    return 0;
}
```

Output:

```
Simple Interest = 600.00
Total Amount = 3600.00
```

12. **How can you solve (-b +√(b2- 4ac)\2a) in the C language?**

```
#include <stdio.h>
#include <math.h>  // for sqrt()
int main() {
    float a, b, c, result;

    // Assign values to a, b, and c
    a = 1;
    b = 4;
    c = 2;

    // Calculate the expression
    result = (-b + sqrt(b * b - 4 * a * c)) / (2 * a);
```

```
    // Output the result
    printf("Result = %.2f\n", result);

    return 0;
}
```

Output:

```
Result = -0.59
```

13. **Predict and write the consequence if modulus operator % is used between two float variables.**

If the modulus operator % is used between two float variables in C, it will result in a compilation error. Because the % operator in C is only defined for integers. It cannot be used with float or double data types.